Don’t Outsource Your Thinking

and other observations on Code Generation Tools

As 2026 rolls on I wanted to write down some thoughts on using Claude Code. I’ve been using these tools for quite a while now and like the first chat models there are similar patterns emerging in how to use these tools. I also hear similar threads from folks who use code generation models in their daily work. A lot of these ideas are still working themselves out in terms of practice, and I wanted to write some of them down before the next wave of model improvements.

Every month provides some updates on how to poke the frontier a bit. Between base models and instruct models, the sft/rlhf detour, the reasoning arc and now rl-vr, model improvements month over month are a given. In early 2023, from a user pov, the key idea was that foundational models will eventually absorb everything around it. Perhaps, maybe not product and UX surfaces. I think this is still true, but building one level above is not a fools errand as people make it out to be. The middle man will eventually be cut out, the productivity of knowledge workers will explode, Anthropic might be more like AWS one day or it will be absorbed into Amazon. You, however, cannot outsource your thinking.

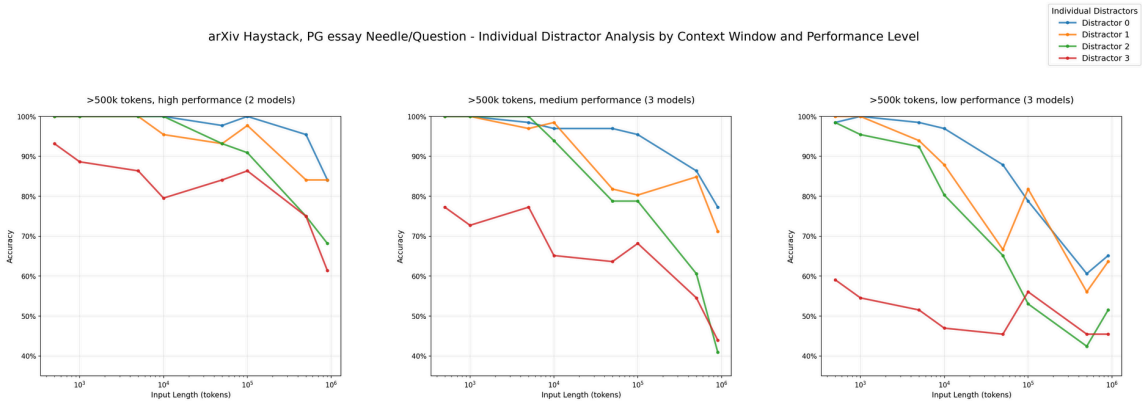
Note #1: Context rot is key to manage well. It is tempting to stuff the context as much as you can, but this will increase the chance of running into hallucinations or bugs, or worse, sycophancy.

Chat models offer a sense of statelessness, they might have caching and memory now, but when you start a new chat the expectation is that they are processing instruments that are biased with every input you give. The larger the model the better it is at In-context Learning. However, be prepared to clean out the context as needed.

CC now helps you with `/clear` and `/catchup`, to make it easier to bootstrap a start state.

Context rot seems to have been coined on orange site by workaccount2. The idea they talked about is things in the context that doesn’t help the LLM. This parlays to how folks typically think about context stuffing and context engineering.

But how do you measure it? Chroma published some work around needle-in-a-haystack style experiments to understand the effect of distractors on finding the needle. Distractors might not typically show up the way they describe it from an everyday developer point of view.



Chroma NIAH with distractors

What usually happens is that in a timeline of chat events, the model will get confused or might forget a task in a long running list of tasks at hand. Worse, a model might ignore the second half of your question in a long chat. Anecdotally in another case, past build logs were “spamming” the models attention toward the real problem. What I like to do in these cases is to simply start over. Or revert to an earlier checkpoint in the chat (chat models and CC offer this feature).

As a mental model, it’s useful to view Agents and their harnesses as a RAG process that needs tuning. With CC skills you’re solidifying a RAG workflow and augmenting it with fixes “tools”,

secemp @secemp9 · Jan 3

for the same reason that we struggle with that ourselves. try to put yourself in the agent/model place, simulate it, say:

- you have a file to use as scratchpad
- you have a task of varying complexity (so no slop or laziness can be allowed to reduce back and forth and mistakes, etc)
- you have the docs/what needs editing/anything that you could need + the model's own knowledge

as long as you can keep the whole file in the model's context, everything is fine. but past a certain time, it become unmanageable. yes it's markdown or whatever and you could use ast to parse it but the problem remains, how would you handle this without sloppy summarization or repeated retrieval that at some point end up eating at the context

if you had only 200k tokens active context with no latent dynamic context, how would you solve it?

now you could argue, but secemp, there is lora, you can do continual training or smth. there is rag and also etc

yes but as you already know, rag is pretty bad, I mean it works but there are so many limitations, aside from the fact we do topk retrieval which isn't enough to get the full picture, we end up still polluting context past a certain amount of information

for lora, yes this somewhat works but you need a self supervised synth data pipeline but more importantly, there is a limit depending on the model size of how much you can do lora before you overwrite the model too much

then you have summarization which depends on the model understanding how to maximize the information density with fewer tokens with regards to the semantics being 1:1 to original data. you can somewhat solve this with RL but it's still not enough in the grand scheme of things, we can do better

---

Note #2: Don’t outsource your thinking to a model, it may or may not be aligned to what you want to do.

If you started using code generation models from around gpt 3.5, you’d be accustomed to a chat interface flow, where you type something into the chat window and then transfer over the generated code into your IDE or editor. You’d also be familiar with tools like Cursor that offer both in-line and file-level completion.

I’ve personally only liked using models that generate snippets that solve a problem or search documentation to solve a problem. I’ve always found it to be more responsible to outsource some amount of grunt work to a model but not the thinking itself.

Here are a few examples,

```
Outsourcing: "here is a dataset, how can I improve response times???"
Better yet: "what is the effect of column A on the outcome, what techniques can I use to
understand the effects?"


Outsourcing: "what is wrong with this code???"
Better yet: "If there are too many requests to my server how can I protect myself?"


Outsourcing: "Can you refactor this code for me???"
Better yet: "Can you analyze this code to tell me what inputs I should care about if I want to
_extract-method_ here?"
```

Notice how the improvements are all focused on one meta idea, “learning to learn”. This is a key point in my opinion. I don’t think this will change with improved model capabilities, because then you’d have to replicate the thinking of a Senior Domain Expert. This then means, you need some sort of embodied intelligence to keep up with the world, or you need some sort of continual learning system that has an optimization function beyond token generation and can keep up it’s intelligence.

Note #3: Outsource grunt work that is predictable and consistent in the task shape, especially to what other developers typically might do.

This note is more familiar to folks who’ve been using models for the last couple of years. I am highlighting it because I still feel models are better at executing small well defined-tasks that match the training data that they have.

The simplest example is when chat models are dealing with fuzziness, if you pass in text and ask it to spot specific strings, they will do a good job. You don’t need to use a vector database for this, esp if you’re thinking about “but what about matching semantically similar terms”? The model is doing the vector search for you over some context window.

What I typically do is to ask the model to generate things like Github action files, terraform scripts, or when I’m feeling too lazy to use jq. I don’t usually need to think about these things, especially when the outcome driven acceptance of the generated code is obvious. If I’m generating a github action then I can just test it. I would of course test it in Staging and not Production first.

CC has skills which I feel is a bigger deal than MCPs. The latter were a bridge or glue towards helping models interact with APIs that are not standardized across context boundaries, and weren’t meant to help LLMs standardize their behaviour. The former however, provides the right abstraction for capturing a “skill”.

If I go from prompting -> add tools -> utilize on-demand scripts, the next logical abstraction is to save this repeated action into a Skill. Here is an example of a pdf parser skill from the standards website. (Note - pdf parsing is by no means a fully solved problem and in a lot of cases still requires custom tuning, as of now.)

Note #4: The harness is what guides the model. Both the model and the harness must be good for the Developer to feel magic.

The obvious limitation with chat models is that they introduce friction into the developer workflow. While the context switching aspect is one part, the friction is copying over code and papering over the rough edges into your IDE. While not exactly a first-world problem it is annoying and hits productivity.

This is where CC comes in with its Agents approach. While CC is a wrapper around Claude, the Harness (ie. the prompts) and the model tuning are substantial. You may or may not have the same experience with other Harnesses like OpenAI Codex or OpenHands.

CC (esp. Opus 4.5 now) is amazing in terms of general intelligence. Pure prompting, with no md files or any other indexes, or additional domain context, *will increase* your productivity, especially as a Senior Developer. The Anthropic blogs for CC are a good starting point to understand where things are.

Note #5: If you don't know what you don't know, you might not be able to ask the model the right questions.

You can do more than just pure prompting and CC along with other tools have come a long way in just this year. I would recommend setting up a CLAUDE.md file using CC's `/init` command. This will go over your codebase and set up basic instructions. As noted in the blog, you should add things to the md file as and when you encounter a situation that has a known repeatable action. For example, if you have a typescript project and you use `pnpm` with `vitest` you might write that down in the md file.

You might also encounter some very interesting issues. For whatever reason, Opus 4.5 doesn't think `vitest` `clearMocks` and `resetMocks` can lead to incorrect unit test setup. This mock setup is a big enough issue that other languages/frameworks have strong opinions about how to write tests, or follow Test Driven Development or Spec Driven Development. Adding them as you encounter them to the md file helps.

Note #6: Grounding the model remains a key aspect, regardless of how the model is exposed to you as a user.

The md file is an entry point for CC that helps ground the model and the harness. Having consistent themes there will help CC do a better job. It might not work if you have multiple personas there. I have a `How does a CTO think` section in my md file that guides high level design choices, including a simple "please structure your changes at a high level before we start working

on it.” It’s good to keep in mind that, grounding in any form (like context engineering) is subject to rot.

Grounding also applies to the process oriented aspects of Software Engineering, for example all tests on an API should handle basic status codes. Adding this to a shared Claude Code guide helps,

Boris Cherny
@bcherny

4/ Our team shares a single CLAUDE.md for the Claude Code repo. We check it into git, and the whole team contributes multiple times a week. Anytime we see Claude do something incorrectly we add it to the CLAUDE.md, so Claude knows not to do it next time.

Other teams maintain their own CLAUDE.md's. It is each team's job to keep theirs up to date.

```
claude-cli $ cat CLAUDE.md
# Development Workflow

**Always use `bun`, not `npm`.**

```sh
# 1. Make changes

# 2. Typecheck (fast)
bun run typecheck

# 3. Run tests
bun run test -- -t "test name"      # Single suite
bun run test:file -- "glob"         # Specific files

# 4. Lint before committing
bun run lint:file -- "file1.ts"     # Specific files
bun run lint                        # All files

# 5. Before creating PR
bun run lint:claude && bun run test
```

1:29 AM · Jan 3, 2026 · **337.1K** Views
```

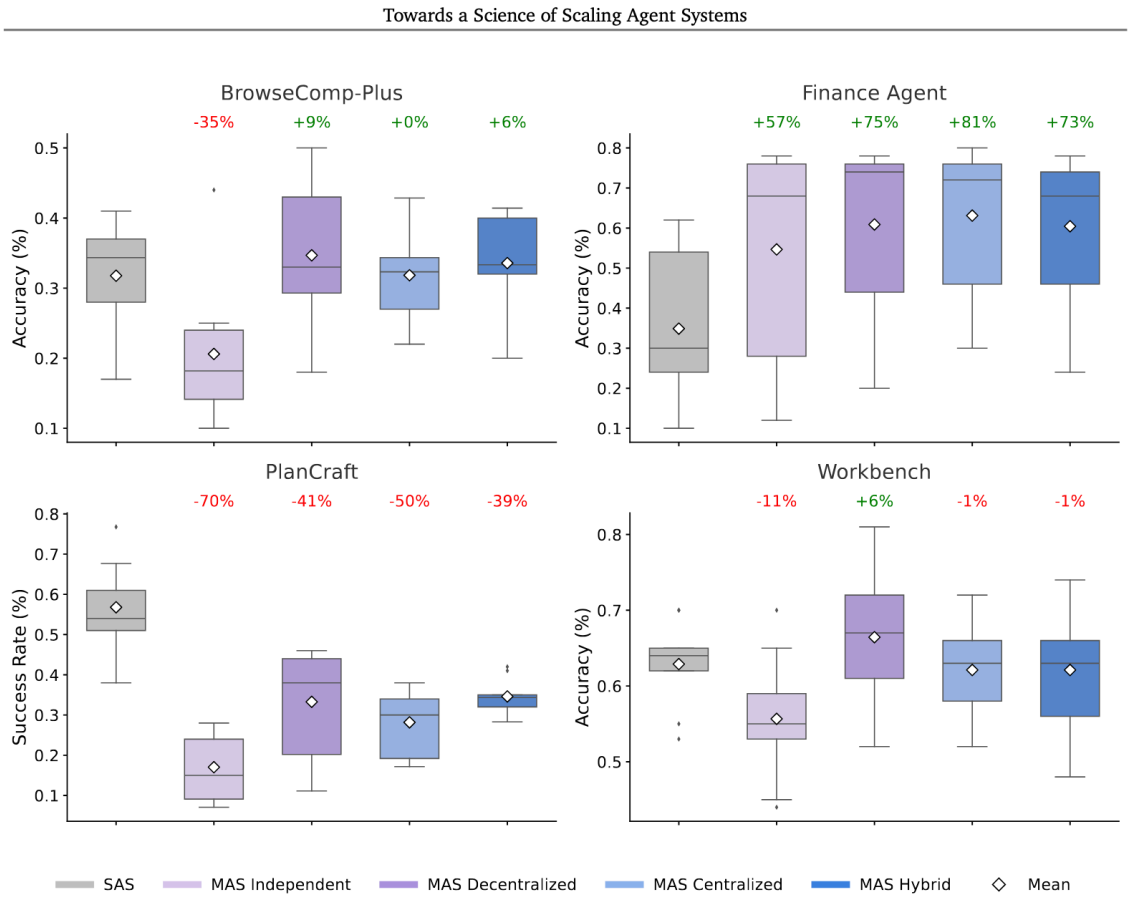
Note #7: Ask the model to plan first. If you don't have a plan you will end up with slop because you can't think through a plan that doesn't exist.

I always ask CC to generate an implementation plan as part of the md file. This helps keep track of tasks and offers a checkpoint to the work done. If for some reason you need to start over, at the very least you have a TODO list that can be executed one after the other.

Note #8: Subagents might take some effort to set up and might not be worth it for your personal productivity. It might make sense to have 1 agent per team member and cut down the overall team size.

I don't use CC with multiple agents. The analogy of a direct report and an agent is apt, if you have 10 sub-agents, that's 10 sources of potential failure. Typically people use CC to spawn an inspector or manager agent that will "manage" employees. This may or may not work, and when it doesn't work you have an explosion in the number of combinations you have to think through to understand why it failed. I'm a fan of the two pizza teams idea, and this core principle has to apply to agents too. This shortcoming will change within the next year.

Depending on the domain, Single-Agent Systems will be beaten by Multi-Agent systems, there are papers out now, for example this one from GDM that compare performance for certain benchmarks,



Multi-agent system architecture

(See also Anthropic's blog on Multi-Agent research systems to understand parallelization in the domain process)

Quoting from the paper,

> *The core difference from Finance Agent lies in task structure, where Finance Agent tasks decompose into parallelizable subtasks (e.g., separate agents can independently analyze revenue trends, cost structures, and market comparisons, then synthesize findings), whereas PlanCraft requires strictly sequential state-dependent reasoning, each crafting action modifies the inventory state that subsequent actions depend upon.*

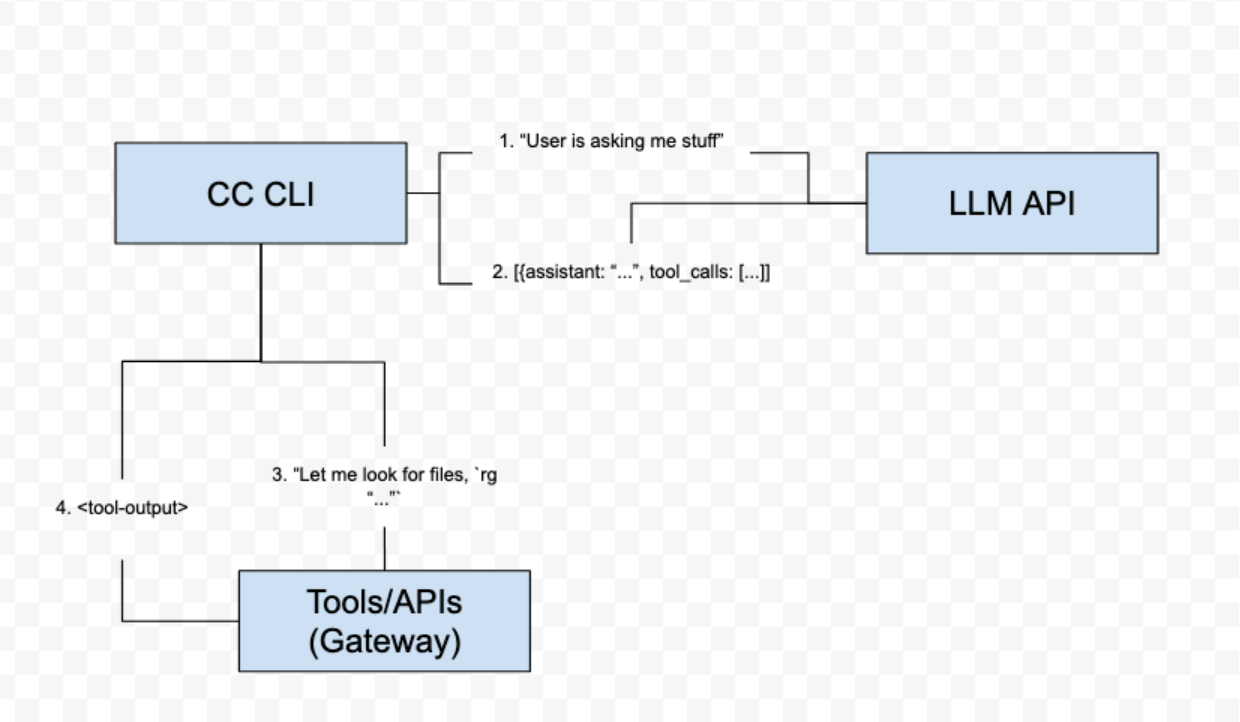
The actionable take away, for the moment, in my opinion, is that Software Engineering is more along the lines of PlanCraft and therefore invariably ends up with logical contention points that need intervention.

Anthropic is now working on the Agents SDK which I think will help with this specific problem of Agent orchestration and coordination. CC is a specific expression of the Agents SDK. I will have to try this out later.

Note #9: Token consumption is a key metric. If a task needs too many tokens to get an output there is an opportunity to optimize.

I haven't set up CC with LSP or use any other semantic code retrieval setup like Serena. Both approaches are a bit new but will help with token consumption over time.

This is how CC is setup to work, orchestrated by the LLM API,



Claude Code architecture

The proxy of token counts for me is just a way to understand what information was passed up to the harness/model. If I let the model run a test suite, I don't need to upload 2-3k of tokens when the suite passes. For failures, I just need to pass in enough of the error for the model to fix the problem, I don't think we can control this aspect of CC directly, except via Checkpoints, or starting a new sub-agent per unit per test.

See Manus's blog for a discussion on KV-Cache hit rates, expressing the same sentiment.

Note #10: Nobody gets promoted for writing unit tests, but if you're a 2-person billion dollar company, you're going to need unit tests. Also, unit tests will never not save your life.

One funny thing I've noticed is that CC takes up a lot of tokens to analyze build outputs (pass or fail). I typically just run the build myself and paste the error, a smaller cheaper model (say via a tool call) might be the right way to automate this step.

When dealing with code, having a test suite is critical. Otherwise, as is often the case, there is too much to do and not enough human mental bandwidth, leading to regressions. Having infra that helps roll-back and roll-forward is critical and a different discussion. From the pov of CC however, having a good test suite to start with is needed to enable the agentic aspects. Without this environment setup, the agent's ability to have autonomy and self-heal is limited from the get go.

Note #11: If you're not thinking declaratively you're going to have a hard time managing AI output.

With AI tools, the cost of greenfield test setup should be very low. If you organize your Test Driven Development correctly, you can utilize CC to write tests and vet them properly. This frame of mind will also help you write your code better. CC and other models tend to write long code blocks. Declarative programming, writing guards upfront, good error handling and logging etc etc, are still critical aspects; CC will easily and quickly set up a Feature Flag for you, you take advantage of this productivity boost. Setting up invariants, using guards, writing good contract tests are still key.

These ideas were key pre-LLM and will continue to be critical.

Note #12: Use a combination of models for sanity checks.

I primarily use CC, but will ask Codex to run code reviews. Having some variety here is important. Ideally you could ask a new instance to do a code review, but that's like asking the PR implementor to review their PR with a fresh mind the next day, it works but that's not the point of a code review. I however make sure to check all the generations as I would with a colleague's work.

Note #13: Greenfield is easy, Brownfield might not be

It is very easy to setup a greenfield project from scratch and go 0->1 in a matter of days and not months. This is critical especially when a lot of design choices and contention points are open to

discussion. Depending on your org YMMV on how quickly you can get the project off the ground. With CC the cost is so low it would be a travesty to not take advantage of it,

Note #14: Treat the Agent like a Pure Function when you need to explore, have specific prompts if needed.

The final note is on Legacy codebases. CC will work real well but it might not have the right context. This is where I try to step in and guide CC assuming it to be a junior engineer starting out on a large legacy codebase - try to understand the code first, make notes, read specific doc versions, write unit tests, make changes.

Acknowledgements

Thanks to secemp and dejavucoder for reading the draft and the conversations!

Some Links for further reading

1. For a slaphappy introduction to CC checkout this guide from my moot dejavucoder.
2. Context Engineering for AI Agents: Lessons from Building Manus
3. Context rot study from Chroma